

UNIVERSIDADE FEDERAL DE SANTA CATARINA
CENTRO TECNOLÓGICO
DEPARTAMENTO DE INFORMÁTICA E ESTATÍSTICA

ARTHUR ERPEN
CAETANO PERUZZO
GABRIEL HERRERA

Relatório sobre a Entrega 1 de Sistemas Operacionais II

Florianópolis
2026

Sumário

1	Introdução	2
2	Arquitetura da Solução	2
2.1	Modelo de Processos e Sistema	2
2.2	Padrões de Projeto	3
2.2.1	Observer	3
2.2.2	Produtor-Consumidor	3
2.3	Pilha de Comunicação em Camadas	4
2.3.1	Communicator	5
2.3.2	Protocol	6
2.3.3	NIC	7
2.3.4	Ethernet Engine (RawSocketEngine)	10
3	Testes Realizados e Métricas Coletadas	11
3.1	Criação dos Veículos e Teste de Comunicação	11
3.2	Benchmark de Latência — RTT Ping-Pong	11
3.3	Estatísticas	12
4	Atendimento aos Requisitos da Primeira Entrega	12

1 Introdução

Este documento detalha a implementação da primeira etapa do projeto de desenvolvimento de uma biblioteca de comunicação para sistemas autônomos críticos. O objetivo central desta entrega é estabelecer a base da comunicação entre sistemas autônomos, modelados como veículos, operando em máquinas virtuais (VMs) distintas emuladas pelo QEMU com arquitetura RISC-V 64-bit.

A solução implementa uma pilha de comunicação em camadas inspirada no modelo OSI, utilizando raw Ethernet sockets (`AF_PACKET`, `SOCK_RAW`) para comunicação direta entre VMs sem dependência da pilha IP. A rede virtual é fornecida pelo QEMU via multicast UDP no host.

2 Arquitetura da Solução

2.1 Modelo de Processos e Sistema

Em conformidade com os requisitos definidos, a arquitetura suporta o seguinte modelo:

- **Veículo:** A classe `Vehicle` representa o macro-objeto. Cada instância é executada em sua própria VM QEMU, sendo responsável por orquestrar seus componentes e a pilha de comunicação associada.
- **Componentes (Sensores, Atuadores):** Cada componente (classe abstrata `Component` e subclasses) é projetado para ser executado em um processo POSIX isolado. A classe `Vehicle` utiliza `fork()` para criar um novo processo para cada componente adicionado.

Ao iniciar a execução, o veículo cria seus componentes como processos filhos, cada um com sua própria instância da pilha de rede:

```
1 pid_t pid = fork();
2 if (pid == 0) {
3     NIC<RawSocketEngine> nic;
4     Vehicle_Protocol protocol(&nic);
5     Channel_Endpoint<Vehicle_Protocol> endpoint(
6         &protocol, protocol.create_address(c.second));
7     c.first->set_endpoint(&endpoint);
8     c.first->set_port(c.second);
9     c.first->run();
10    _exit(0);
11 }
```

Listing 1: Trecho de `Vehicle::run()` — fork com pilha de comunicação

A implementação utiliza o padrão *Dependency Injection* para isolar a lógica do componente da infraestrutura de comunicação. Conforme observado no código acima, no momento do `fork()`, o processo filho instancia uma nova pilha de rede completa (`NIC`, `Protocol`).

Para permitir a comunicação, é utilizado o `Channel_Endpoint`, uma especialização da interface abstrata `Communication_Endpoint`. Este funciona como um *Adapter* que encapsula um `Communicator` parametrizado com o protocolo específico da rede (`Vehicle_Protocol`). O componente, por sua vez, armazena apenas um ponteiro para a interface base:

- **Polimorfismo e Generalização:** A classe `Component` depende apenas da interface `Communication_Endpoint`, tornando-a agnóstica quanto ao meio físico. Na Etapa 1, ela recebe um `Channel_Endpoint` que provê acesso à rede via sockets raw; em etapas futuras, este poderia ser substituído por implementações diferentes.
- **Especialização do Endpoint:** O `Channel_Endpoint<Channel>` mapeia as chamadas genéricas `send()` e `receive()` para os métodos correspondentes do `Communicator`, utilizando o endereço e a porta criados especificamente para aquele processo filho.

2.2 Padrões de Projeto

Neste projeto, foram aplicados dois padrões de projeto que estruturam a comunicação entre as camadas e permitem um funcionamento assíncrono e desacoplado:

2.2.1 Observer

O padrão Observer é utilizado para a demultiplexação assíncrona de dados entre as camadas da pilha de comunicação, em dois níveis:

- **NIC → Protocol (Conditional_Data_Observer):** A NIC atua como `Observed`, notificando os `Protocols` (`Observers`) quando um frame com um `EtherType` correspondente é recebido. A condição de filtragem é o `EtherType` do frame.
- **Protocol → Communicator (Concurrent_Observer):** O `Protocol` atua como `Observed`, notificando os `Communicators` (`Observers`) quando um pacote destinado a uma porta específica chega. A condição de filtragem é a `Port` de destino.

Dessa forma, a notificação ocorre em cascata, permitindo que apenas os interessados em determinado tipo de dado ou porta sejam despertados.

2.2.2 Produtor-Consumidor

O padrão Produtor-Consumidor é aplicado entre a thread de rede e a thread da aplicação, desacoplados por meio do `Concurrent_Observer` e do `Communicator`:

- **Produtor:** a thread de rede, ao receber um pacote, invoca o método `update()` do `Communicator`, que adiciona o ponteiro do buffer em uma fila e sinaliza um semáforo (`_semaphore.v()`).
- **Consumidor:** a thread da aplicação, ao chamar `receive()`, bloqueia no semáforo (`_semaphore.p()`). Ela só é desbloqueada quando novos dados são produzidos e sinalizados pela thread de rede.

Esse padrão garante que a aplicação só processe mensagens disponíveis, enquanto a thread de rede continua operando de forma independente. Os diagramas abaixo detalham esse comportamento em ambas as threads:

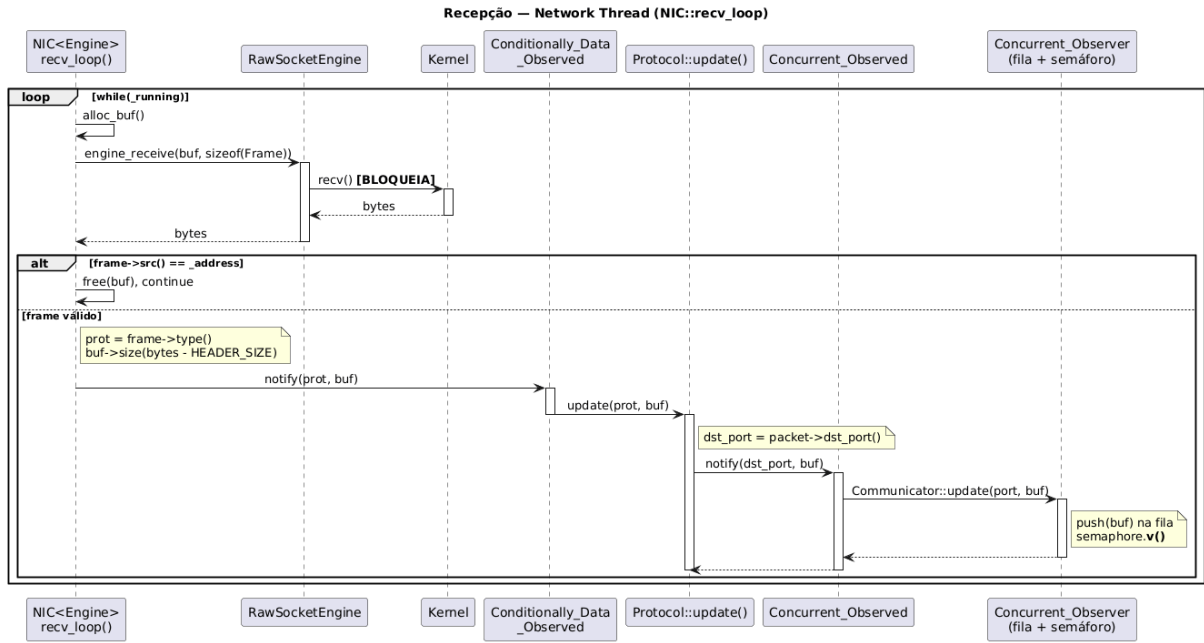


Figura 1: Diagrama de Sequência: Recepção na Thread de Rede (Produtor)

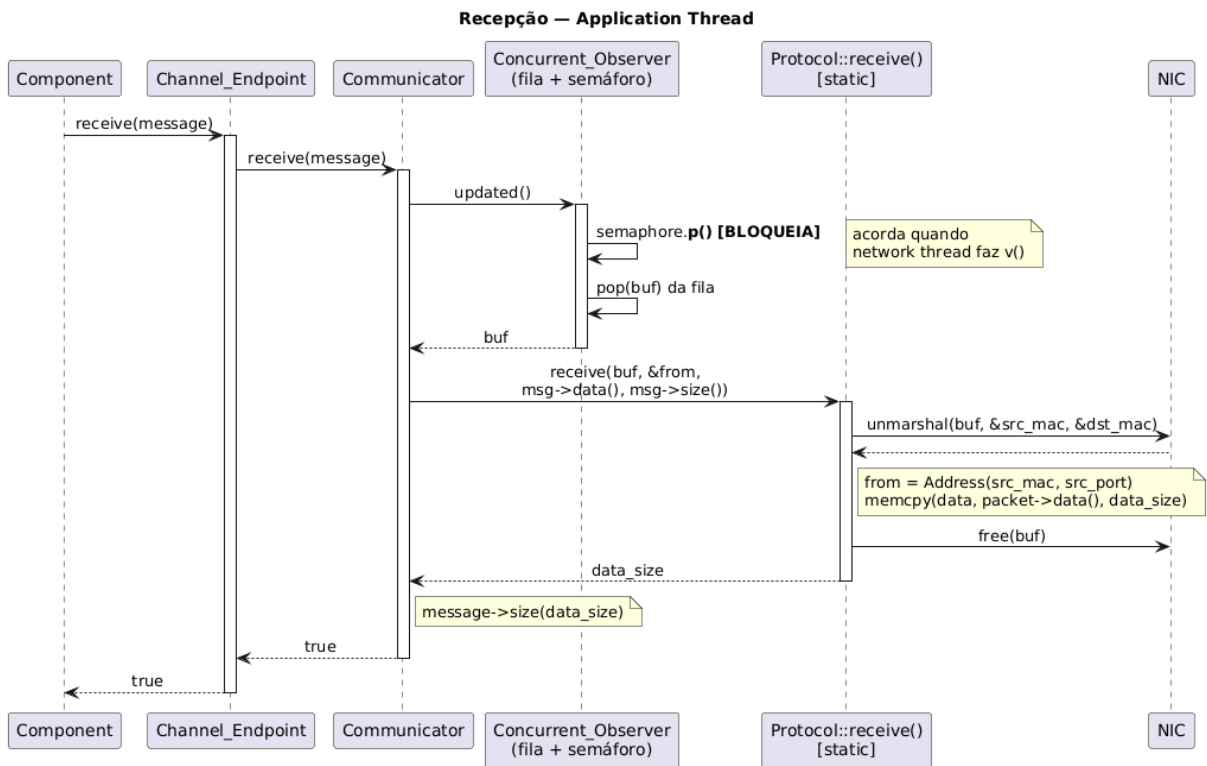


Figura 2: Diagrama de Sequência: Recepção na Thread da Aplicação (Consumidor)

2.3 Pilha de Comunicação em Camadas

A comunicação do projeto é estruturada em uma pilha de quatro camadas principais.

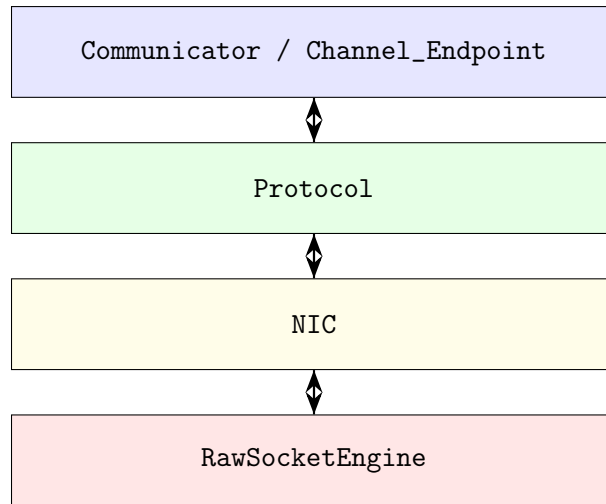


Figura 3: Hierarquia da pilha de comunicação

2.3.1 Communicator

O Communicator fornece a API de comunicação diretamente para a aplicação, funcionando como um endpoint que abstrai toda a complexidade das camadas inferiores. Ele disponibiliza um método para envio e outro para recebimento, ambos operando sobre a classe `Message`, que representa o conteúdo útil transmitido e recebido.

A classe `Message` mantém um payload em forma de sequência de bytes com tamanho máximo de 1400 bytes (garantindo que não ultrapasse o MTU da Ethernet após os cabeçalhos).

O Communicator é parametrizado por um tipo `Channel` que nesta entrega é instanciado como `Protocol`. No envio, ele repassa a referência da `Message` ao método `send` do channel, junto com o endereço de origem e o de destino (broadcast):

```

1 bool send(const Message * message) {
2     return (_channel->send(_address,
3         Channel::Address::broadcast(_address.port()),
4         message->data(), message->size()) > 0);
5 }
  
```

Listing 2: Método `send` do Communicator

No recebimento, a aplicação chama `receive` passando uma `Message` vazia. O Communicator atua como observador concorrente do channel (a porta do Communicator é a condição observada). Quando o channel recebe um pacote destinado a essa porta, ele notifica o Communicator via `update()`, que desbloqueia a chamada `receive()`:

```

1 bool receive(Message * message) {
2     Buffer * buf = Observer::updated(); // bloqueia ate notificacao
3     if (!buf) return false;
4
5     typename Channel::Address from;
6     int size = _channel->receive(buf, &from,
7         message->data(), message->size());
8     message->size(size);
9     return (size > 0);
10 }
  
```

2.3.2 Protocol

A classe `Protocol` é responsável pela implementação de um protocolo de transporte simples sobre o Ethernet. Ela é a camada de comunicação intermediária entre a NIC e a aplicação (`Communicator`). Por conta disso, a classe `Protocol` é um `observer` de NIC e é observada pelas aplicações (`Communicators`).

O `Protocol` define um `Packet` que contém um cabeçalho (`Header`) com porta de origem e porta de destino, seguido pelo payload da mensagem:

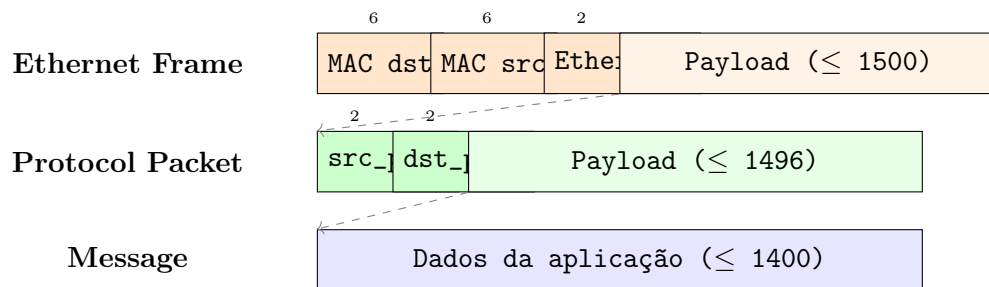


Figura 4: Formato do frame Ethernet com encapsulamento do Protocol

No fluxo de envio, o `Communicator` chama `Protocol::send()` que: (1) aloca um buffer na NIC com tamanho do cabeçalho + payload, (2) preenche as portas de origem e destino no `Header`, (3) copia os dados da mensagem para o payload do pacote, e (4) envia via `NIC::send()`. O fluxo de envio completo, englobando todas as camadas, pode ser visualizado no diagrama abaixo:

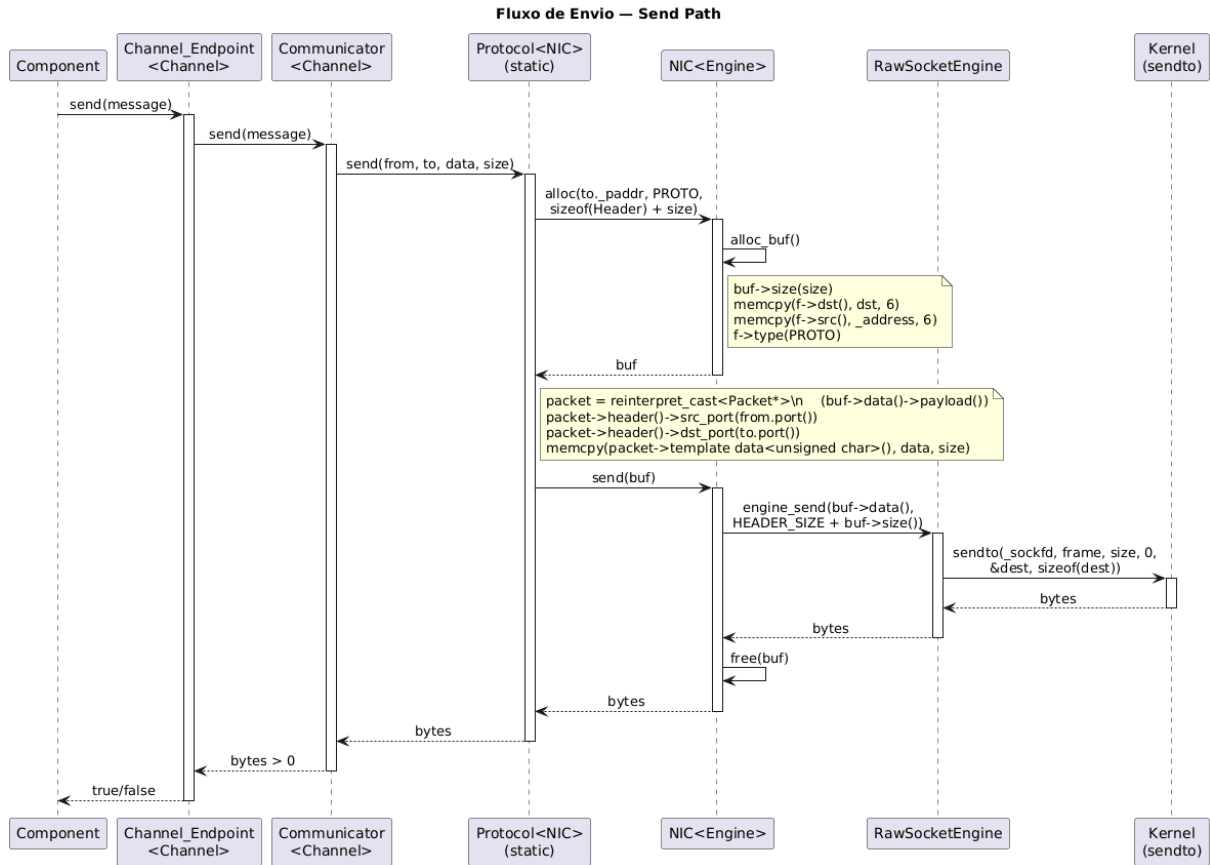


Figura 5: Visão Geral do Fluxo de Envio (Send Path)

No fluxo de recepção, quando a NIC recebe um frame com EtherType 0x8888, notifica o Protocol via `update()`. O Protocol extrai a porta de destino do cabeçalho do pacote e notifica o Communicator correspondente via `Concurrent_Observed::notify()`:

```

1 void update(typename NIC::Protocol_Number prot, Buffer *buf) {
2     if (!buf) return;
3     if (buf->size() < sizeof(Header)) {
4         _nic->free(buf); return;
5     }
6     Packet *packet = reinterpret_cast<Packet *>(
7         buf->data()->payload());
8     bool notified = _observed.notify(
9         packet->header()->dst_port(), buf);
10    if (!notified)
11        _nic->free(buf);
12 }
  
```

Listing 4: Método `update` do Protocol — demultiplexação por porta

O Protocol é implementado como singleton (membro estático `_instance`) para que os métodos estáticos `send()` e `receive()` possam acessar a instância da NIC.

2.3.3 NIC

A NIC abstrai os detalhes da Engine para a camada de comunicação, sendo responsável por gerenciar a alocação de buffers com Frames Ethernet para envio e recepção de dados.

A estrutura do frame, incluindo cabeçalhos, payload e constantes relacionadas, é definida na classe `Ethernet` herdada pela `NIC`.

Para envio, a `NIC` mantém um pool de buffers (vetor estático + free-list baseada em `std::stack`), cujo tamanho é definido em `Traits` (8 buffers de envio + 16 de recepção). Quando a camada superior deseja transmitir dados, ela utiliza a interface da `NIC` para alocar um buffer, informando o endereço físico de destino, o protocolo (`EtherType`) e o tamanho:

```
1 Buffer<Ethernet::Frame> *alloc(Address dst,
2   Protocol_Number prot, unsigned int size) {
3   Buffer<Ethernet::Frame> *buf = alloc_buf();
4   if (!buf) return nullptr;
5   buf->size(size);
6   Ethernet::Frame &frame = *buf->data();
7   memcpy(frame.src().raw(), _address.raw(), Address::LENGTH);
8   memcpy(frame.dst().raw(), dst.raw(), Address::LENGTH);
9   frame.type(prot);
10  return buf;
11 }
```

Listing 5: Alocação de buffer na `NIC`

A função `alloc_buf()` é thread-safe (protegida por mutex) e retorna `nullptr` se não houver buffers disponíveis — nesse caso o pacote é descartado, pois a simulação de colisões e retransmissões ainda não faz parte do escopo. O ciclo de vida e a máquina de estados dos buffers alocados para envio são ilustrados a seguir:

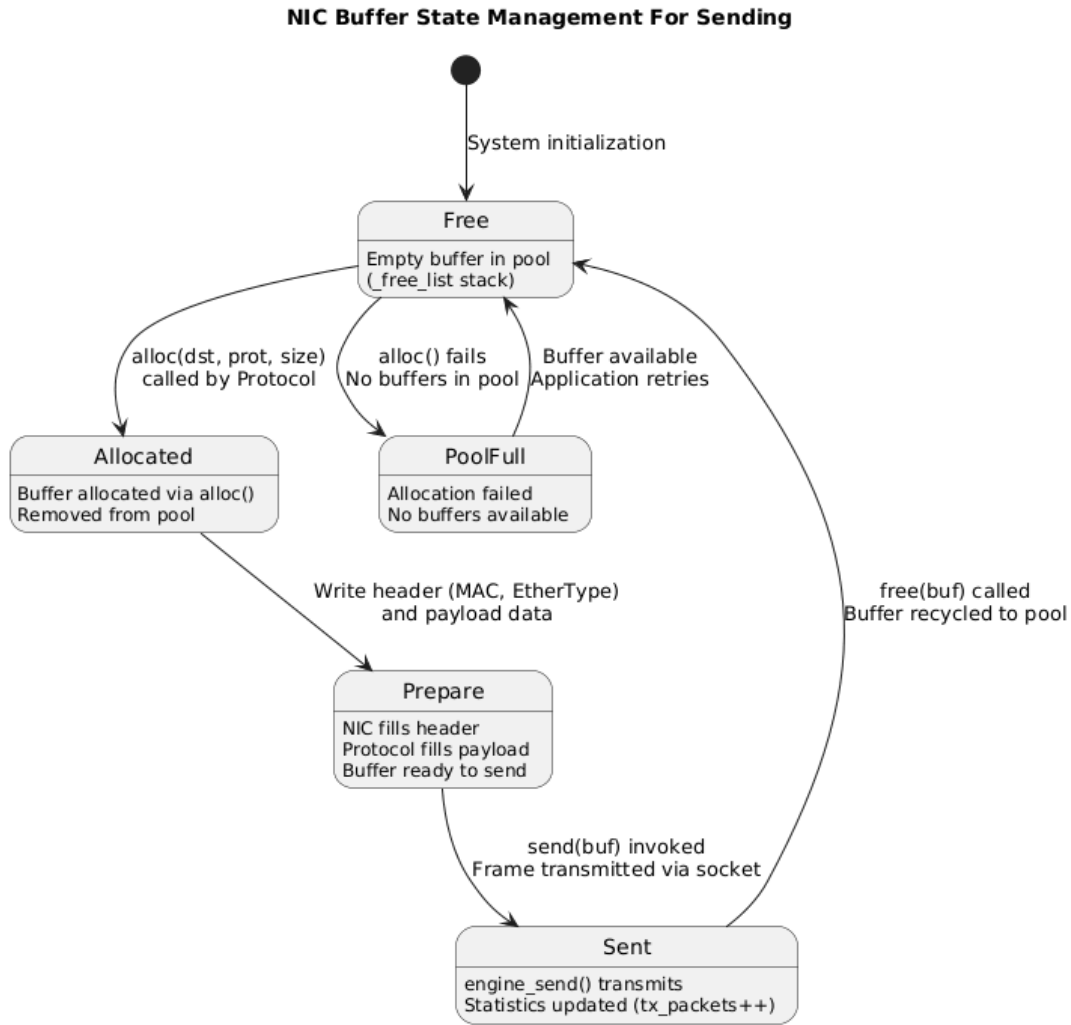


Figura 6: Máquina de Estados: Gerenciamento de Buffers da NIC para Envio

No caminho inverso, a NIC cria uma thread de recepção, chamada sempre que bytes chegam através da engine, essa thread aloca um buffer, copia os dados e notifica os observadores interessados no protocolo especificado:

```

1 void recv_loop() {
2     while (_running) {
3         unsigned char tmp[Ethernet::MTU + Ethernet::HEADER_SIZE];
4         int n = Engine::engine_receive(tmp, sizeof(tmp));
5         if (n <= 0) continue;
6         Buffer<Ethernet::Frame> *buf = alloc_buf();
7         if (!buf) { _statistics.rx_packets++; continue; }
8         buf->size(n - Ethernet::HEADER_SIZE);
9         memcpy(buf->data(), tmp, n);
10        Ethernet::Protocol ethertype = buf->data()->type();
11        if (!Observed::notify(ethertype, buf))
12            free(buf);
13    }
14 }

```

Listing 6: Loop de recepção da NIC (simplificado)

O gerenciamento do ciclo de vida dos buffers destinados ao recebimento de pacotes ocorre conforme a máquina de estados abaixo:

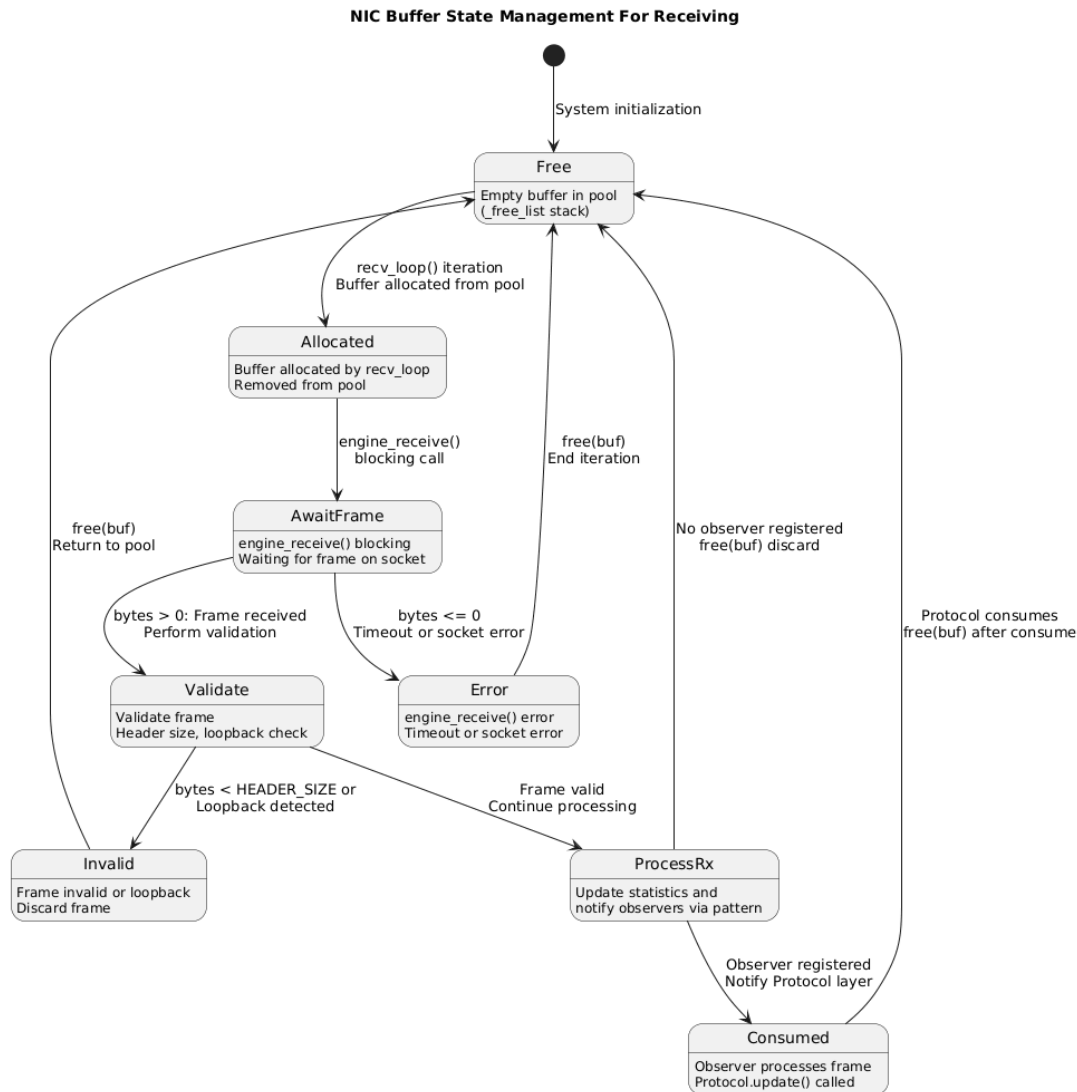


Figura 7: Máquina de Estados: Gerenciamento de Buffers da NIC para Recepção

2.3.4 Ethernet Engine (RawSocketEngine)

A Engine representa a camada mais baixa da pilha de comunicação. Sua principal responsabilidade é abstrair o uso de raw sockets no envio e recebimento de bytes diretamente pela placa de rede.

Na inicialização, o socket é criado sobre a interface de rede `eth0` (fornecida pelo ambiente QEMU), com endereço de broadcast configurado. Esse processo também permite à Engine descobrir e expor publicamente o endereço MAC da máquina local, o que possibilita à camada superior identificar a origem física dos pacotes.

```

1 void engine_init(const char* interface_name) {
2     _sockfd = socket(AF_PACKET, SOCK_RAW, htons(ETH_P_ALL));
3     _ifindex = if_nametoindex(interface_name);
4     struct sockaddr_ll addr = {};
5     addr.sll_family = AF_PACKET;

```

```

6     addr.sll_protocol = htons(ETH_P_ALL);
7     addr.sll_ifindex   = _ifindex;
8     bind(_sockfd, (struct sockaddr *)&addr, sizeof(addr));
9 }

```

Listing 7: Inicialização do socket raw

Para envio, a classe expõe um método que transmite um buffer de bytes via socket com `sendto()`. Para a recepção, ela disponibiliza um método que encapsula a chamada bloqueante `recv()`, delegando à camada superior a responsabilidade de gerenciar o laço de escuta.

3 Testes Realizados e Métricas Coletadas

Esta seção apresenta os testes realizados e as métricas coletadas para avaliar o funcionamento e desempenho da solução.

3.1 Criação dos Veículos e Teste de Comunicação

São criadas 5 VMs QEMU RISC-V 64-bit, cada uma executando uma instância do binário de teste compilado estaticamente com `riscv64-linux-gnu-g++`. Cada veículo é identificado pelo parâmetro `so2.vm_id=N` passado via `/proc/cmdline` do kernel.

O teste principal (`mesh-concurrent`) valida a comunicação em malha completa: todas as 5 VMs enviam e recebem mensagens simultaneamente. Cada VM executa 2 rounds de 3 mensagens (6 envios), e recebe 24 mensagens das outras 4 VMs ($4 \times 6 = 24$). O payload de cada mensagem contém metadados que permitem validação:

```

1 // formato: "mesh-concurrent:r{round}:vm{sender_id}:m{seq}"
2 snprintf(buffer, size, "%s:r%d:vm%d:m%d",
3          config.label, round, sender_id, sequence);

```

Listing 8: Formato do payload de teste

A verificação garante: (1) não há duplicatas, (2) não há mensagens próprias, (3) todos os metadados são válidos, e (4) todas as mensagens esperadas foram recebidas.

3.2 Benchmark de Latência — RTT Ping-Pong

Para medição de latência, optamos por uma abordagem de RTT (Round-Trip Time) com ping-pong entre duas VMs, ao invés de captura de pcap.

Motivação: A abordagem de pcap (captura por VM via `filter-dump` do QEMU) apresenta um problema fundamental de sincronização. Cada VM QEMU é um processo separado no host, e seus timestamps não são comparáveis entre si — o kernel pode agendar o processo receptor antes do emissor, resultando em timestamps de recepção anteriores aos de envio.

A abordagem RTT resolve isso medindo ida e volta com o **mesmo clock** local:

```

1 for (int seq = 1; seq <= config.sample_count; ++seq) {
2     unsigned long long start_us = monotonic_raw_us();
3     send_message("ping", seq);
4     receive_expected("pong", RESPONDER_VM_ID, seq);
5     unsigned long long end_us = monotonic_raw_us();

```

```

6 |     _samples_us.push_back(end_us - start_us);
7 | }

```

Listing 9: Medição de RTT no initiator (VM1)

O clock utilizado é `CLOCK_MONOTONIC_RAW`, que não é afetado por ajustes de NTP. As amostras são armazenadas em memória e impressas apenas ao final da execução, conforme orientado pelo professor.

Arquitetura do teste: VM1 (initiator) envia pings sequenciais para VM2 (responder), que responde com pongs. VM3–VM5 geram tráfego de fundo para simular carga realista na rede. A latência unidirecional estimada é $RTT/2$.

3.3 Estatísticas

Os valores de latência serão preenchidos após a execução do benchmark com 10 repetições via `make measure-rtt-10`. Cada repetição coleta 20 amostras de RTT (ida e volta).

Métrica	Valor
Amostras totais	200 (10×20)
RTT médio	<i>6.3557 ms</i>
RTT mínimo	<i>2.5820 ms</i>
RTT máximo	<i>104.0710 ms</i>
Latência estimada ($RTT/2$)	<i>3.1779 ms</i>

Tabela 1: Estatísticas de latência do protocolo 0x8888

4 Atendimento aos Requisitos da Primeira Entrega

Esta seção resume como cada requisito específico da primeira entrega foi cumprido pela implementação atual:

- **Desenvolvimento em C++ com Bibliotecas Padrão:** Todo o código foi desenvolvido em C++, utilizando exclusivamente recursos da biblioteca padrão do C++ e da biblioteca padrão do C (libc) para interações com o sistema operacional POSIX.
- **API Minimalista:** A arquitetura implementada com as classes `Communicator`, `Protocol`, `NIC` e `RawSocketEngine` constitui a versão inicial da API, suportando a comunicação entre sistemas. A aplicação conhece apenas a interface do `Communicator` (via `Communication_Endpoint`), evitando detalhes de implementação das demais camadas.
- **Sistema Autônomo como VM QEMU:** A classe `Vehicle` representa o macro-objeto. A arquitetura foi projetada para que a aplicação seja compilada e executada dentro de uma VM QEMU RISC-V 64-bit. A comunicação via `RawSocketEngine` depende da interface de rede virtual fornecida pelo ambiente QEMU.
- **Componente como Processo POSIX:** Este requisito é explicitamente atendido na implementação do método `Vehicle::run()`. Para cada Component adicionado ao veículo, a chamada de sistema `fork()` é utilizada para criar um novo processo

filho. O código do componente (`c->run()`) é então executado exclusivamente dentro desse novo processo, garantindo o isolamento de memória e o modelo de um componente por processo.

- **Comunicação entre VMs com Raw Sockets:** A classe `RawSocketEngine` implementa a comunicação utilizando raw sockets (`AF_PACKET`, `SOCK_RAW`), o que permite o envio e recebimento de frames Ethernet brutos, possibilitando a comunicação direta entre VMs sem a necessidade de uma pilha IP.
- **Comunicação via Broadcast em Rede Virtual:** A implementação garante a comunicação via broadcast. A classe `Communicator` utiliza um endereço de destino `Channel::Address::broadcast()`, que é definido no Protocol como o endereço MAC `FF:FF:FF:FF:FF:FF`. A camada NIC então constrói o cabeçalho do frame Ethernet usando este endereço de destino, assegurando que os pacotes sejam recebidos por todas as VMs conectadas na rede virtual.
- **Padrão Observer e Eventos Assíncronos:** A recepção assíncrona é garantida pela NIC, que cria uma thread dedicada para bloquear no `recv()` da engine enquanto aguarda novos bytes recebidos da rede. Quando um pacote chega, este evento é propagado para cima na pilha usando o padrão Observer:
 1. A NIC notifica o Protocol com base no `EtherType`.
 2. O Protocol notifica o Communicator com base na Port.
 3. O Communicator recebe a notificação em seu método `update()`
- **Cenário de Teste:** 5 VMs criadas; todas atuam simultaneamente como emissoras e receptoras de mensagens em malha completa, validando envios e recebimentos com metadados.
- **Submissão do Projeto e Makefile:** O requisito é cumprido com base no Makefile na raiz, além dos relatórios e documentação na pasta `doc/`. O repositório GitHub foi compartilhado com o professor.